

PROJECT DOCUMENTATION

AI Video Meeting Assistant

A Retrieval-Augmented Generation (RAG) based tool to transcribe, summarise and interact with meeting recordings

Submitted as part of College Project Submission

Domain: Artificial Intelligence / Natural Language Processing

Year: 2026

Student Name: Hafsa Fayaz

Roll No / Enrolment No: 263611

SYAIML

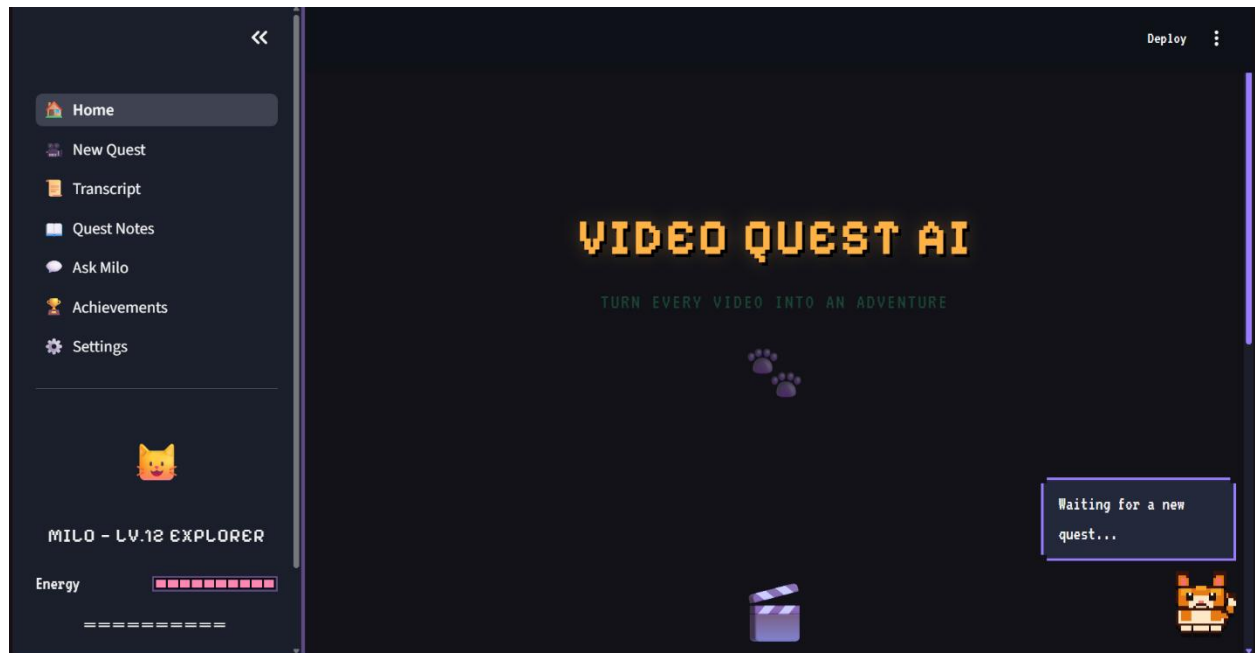


Table of Contents

1. Abstract.....	3
2. Introduction.....	4
2.1 Background · 2.2 Motivation · 2.3 Objectives · 2.4 Scope of the Project.....	4
3. Literature Survey & Reference Work	5
4. Technology Stack	5
5. System Architecture.....	7
6. Features / Functional Description	8
7. Module-wise Implementation Details.....	8
8. Working Methodology.....	9
9. Testing.....	9
10. Advantages and Limitations	10
11. Future Scope.....	10
12. Conclusion	11
13. References.....	12

1. Abstract

In today's fast-paced professional environment, meetings generate large volumes of spoken information that is often difficult to revisit, summarise, or act upon. The AI Video Meeting Assistant is a Retrieval-Augmented Generation (RAG) based application that automatically transcribes, summarises, and enables interactive querying of meeting recordings. The tool accepts either a YouTube video URL or a local audio/video file, transcribes the speech using OpenAI Whisper for English content and Sarvam AI for Hindi and Hinglish content, and then processes the transcript through a LangChain LCEL pipeline powered by the Mistral AI language model. The system produces a structured meeting summary, extracts action items with owners and deadlines, identifies key decisions, and lists open questions and follow-ups. Using HuggingFace embeddings and ChromaDB as a vector store, the application also allows users to chat directly with the meeting content through a Retrieval-Augmented Generation interface. The entire experience is delivered through a custom-styled Streamlit web application, and the final report can be exported as a PDF or TXT document. This project demonstrates the practical integration of speech recognition, large language models, vector databases, and modern pipeline orchestration to solve a real-world productivity problem.

2. Introduction

2.1 Background

Organisations conduct a large number of meetings every week, many of which are recorded but rarely revisited in detail. Manually reviewing recordings to extract summaries, decisions, and action items is time-consuming and error-prone. With advances in speech recognition and large language models (LLMs), it is now possible to automate this process end-to-end, converting raw audio or video into structured, searchable, and actionable text.

2.2 Motivation

Multilingual teams, particularly in India, frequently conduct meetings in a mix of Hindi and English (Hinglish). Most existing transcription tools are optimised for English and perform poorly on Hindi or code-mixed speech. This project was motivated by the need for a single tool that can handle both English and Hindi/Hinglish meetings, generate meaningful summaries, and allow users to interactively query the meeting content instead of reading a long transcript.

2.3 Objectives

- To build a tool that converts any meeting recording (YouTube link or local file) into an accurate text transcript.
- To support both English (via local Whisper) and Hindi/Hinglish (via Sarvam AI) transcription.
- To automatically generate a bullet-point summary, action items, key decisions, and open questions from the transcript.
- To implement a Retrieval-Augmented Generation (RAG) chat interface so users can ask questions about the meeting.
- To provide a clean, styled web interface using Streamlit and export the final report as PDF or TXT.

2.4 Scope of the Project

The system is designed for individuals, teams, and organisations that want a fast, low-cost way to convert meeting recordings into structured knowledge. It focuses on transcription, summarisation, structured information extraction, and conversational querying, and is built primarily using free and locally-hosted components to keep operating costs minimal.

3. Literature Survey & Reference Work

The core AI pipeline logic for this project — including the use of LangChain LCEL for orchestration, Mistral AI as the language model, and ChromaDB with HuggingFace embeddings for Retrieval-Augmented Generation — was studied and adapted from a publicly available YouTube tutorial. The tutorial's implementation approach was used as a reference and foundation, which was then extended and customised for this project, including the addition of Hindi/Hinglish transcription support via Sarvam AI and a custom-styled Streamlit interface built with style.css.

- **Reference Tutorial:** AI Video Assistant using RAG (YouTube)

Link: <https://www.youtube.com/watch?v=xlYJhtL0qbQ>

Beyond the reference tutorial, the broader landscape of meeting-assistant tools (such as Otter.ai and Fireflies.ai) was considered. These commercial tools offer strong English transcription and summarisation but generally lack robust support for Hindi/Hinglish speech and do not allow full customisation of the underlying pipeline. This project addresses that gap by combining free, locally-hosted, and open-source components into a single customisable pipeline.

3.1 Limitations of Existing Systems

- Most commercial transcription tools are subscription-based and expensive for individual or student use.
- Limited or inaccurate support for Hindi and Hinglish (code-mixed) speech.
- Closed-source pipelines that cannot be customised or extended.
- Lack of an integrated, conversational (chat-based) way to query meeting content.

3.2 Proposed Solution

The proposed system combines a free, local speech-to-text engine (Whisper) for English with a specialised API (Sarovam AI) for Hindi/Hinglish, an open pipeline orchestration framework (LangChain LCEL), a free LLM API (Mistral AI), and a local vector database (ChromaDB) to deliver an end-to-end, low-cost, and extensible meeting assistant with a conversational RAG interface.

4. Technology Stack

The application is built entirely in Python and combines local, open-source, and free-tier cloud components to minimise cost while maintaining high accuracy. The table below summarises each component of the stack and its role in the system.

Component	Technology Used	Purpose
Programming Language	Python	Core language for the entire application
Speech-to-Text (English)	OpenAI Whisper (local, free)	Transcribes English meeting audio/video locally without API cost

Component	Technology Used	Purpose
Speech-to-Text (Hindi/Hinglish)	Sarvam AI	Accurately transcribes Hindi and Hinglish meeting audio
Orchestration Framework	LangChain (LCEL)	Builds a modern, composable pipeline connecting transcription, LLM and retrieval
Large Language Model	Mistral AI (free API)	Generates summaries, extracts action items, decisions and answers chat queries
Vector Database	ChromaDB	Stores embedded transcript chunks for Retrieval-Augmented Generation (RAG)
Embeddings	HuggingFace Embeddings (local, free)	Converts transcript text into vector representations for semantic search
Frontend / UI	Streamlit + custom style.css	Provides an interactive, styled web interface for the user
Report Export	Python (PDF/TXT generation)	Exports the final meeting report as a downloadable PDF or TXT file

4.1 Why This Stack Was Chosen

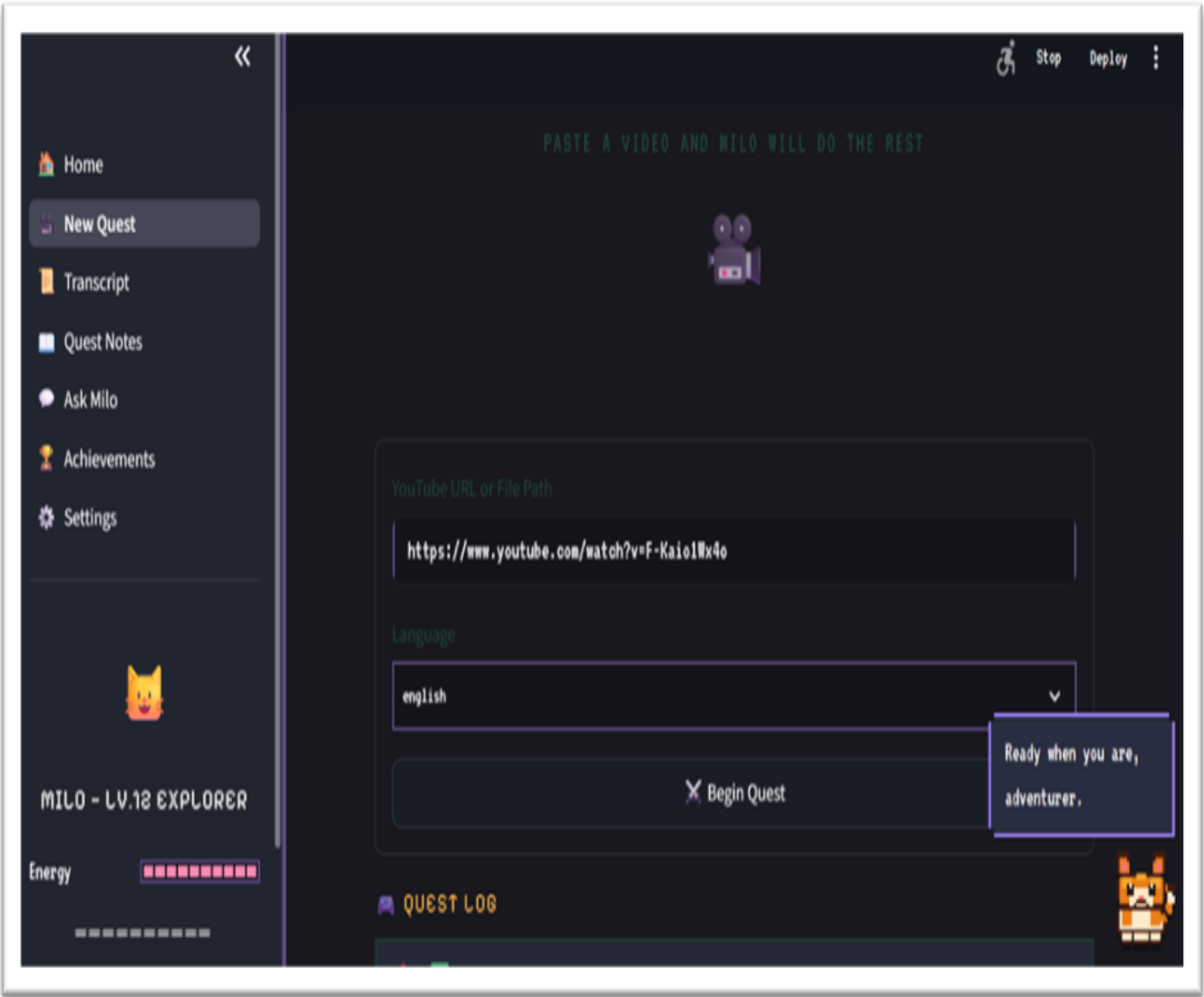
- Local Whisper avoids per-minute transcription costs for English audio and works fully offline.
- Sarvam AI was chosen specifically for its strong performance on Hindi and Hinglish speech, which general-purpose models handle poorly.
- LangChain LCEL (LangChain Expression Language) allows the transcription-to-summary-to-RAG pipeline to be built as a clean, composable, declarative chain.
- Mistral AI provides a capable, free-tier LLM API, avoiding the cost of premium models for a student/college project.
- ChromaDB is a lightweight, easy-to-embed vector database ideal for local RAG applications.
- HuggingFace Embeddings run locally and are free, removing dependency on paid embedding APIs.
- Streamlit enables rapid development of an interactive web UI, further enhanced here with a custom style.css for a polished, non-default look.

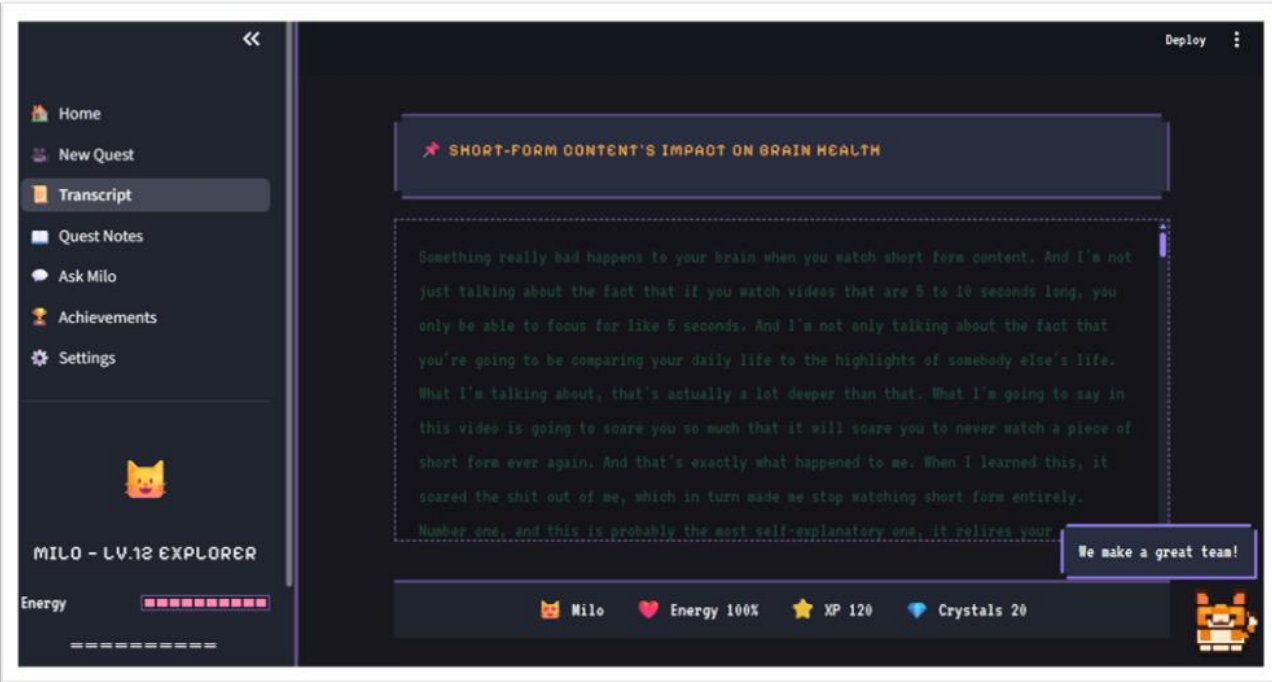
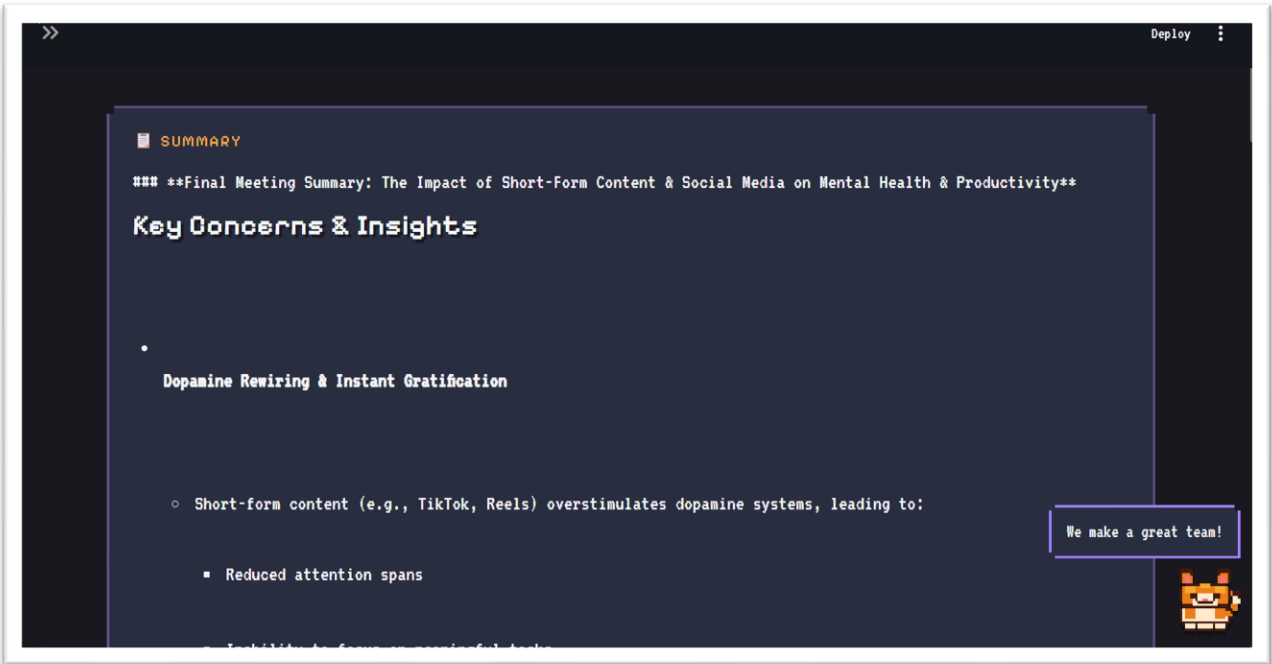
5. System Architecture

The system follows a linear, pipeline-based architecture. Input audio/video is first converted to text, the text is then processed by the LLM pipeline to generate structured outputs, and simultaneously indexed into a vector store to power the conversational RAG chat feature.

5.1 Architecture Flow Description

- **Input Stage:** User provides a YouTube URL or uploads an audio/video file through the Streamlit interface.
- **Transcription Stage:** The system detects the input language — English content is transcribed locally using OpenAI Whisper; Hindi/Hinglish content is sent to the Sarvam AI API for transcription.
- **Processing Stage:** The resulting transcript is passed through a LangChain LCEL pipeline, which sends structured prompts to the Mistral AI model.
- **Extraction Stage:** Mistral AI generates the meeting summary, action items (with owner and deadline), key decisions, and open questions/follow-ups.
- **Indexing Stage:** The transcript is chunked, converted into vector embeddings using HuggingFace Embeddings, and stored in ChromaDB.
- **RAG Chat Stage:** User queries are embedded, relevant transcript chunks are retrieved from ChromaDB, and Mistral AI generates a context-aware answer.
- **Export Stage:** The final structured report can be downloaded as a PDF or TXT file.





6. Features / Functional Description

Feature	Description
Flexible Input	Accepts any YouTube URL or a local audio/video file as input
English Transcription	Uses local Whisper AI to transcribe English meetings, keeping the process offline and free
Hindi & Hinglish Transcription	Uses Sarvam AI to accurately transcribe Hindi and Hinglish meetings
Meeting Summarisation	Condenses the full meeting transcript into clear, bullet-point summaries
Action Item Extraction	Identifies action items along with the responsible owner and deadline
Key Decision Extraction	Pulls out the important decisions made during the meeting
Open Questions & Follow-ups	Surfaces unresolved questions and follow-up items for the team
Conversational RAG Chat	Lets the user chat with the meeting transcript using Retrieval-Augmented Generation and ChromaDB
Report Export	Allows the complete report to be exported as a PDF or TXT file

7. Module-wise Implementation Details

7.1 Input & Transcription Module

This module accepts either a YouTube URL (from which audio is extracted) or a directly uploaded audio/video file. Based on the detected language, the module routes the audio to one of two transcription engines: OpenAI Whisper (running locally, at no cost) for English content, or the Sarvam AI API for Hindi and Hinglish content. The output of this module is a clean, timestamped text transcript.

7.2 Summarisation & Extraction Module

The transcript is passed into a LangChain LCEL pipeline. LCEL chains are used to structure the prompt sent to Mistral AI so that it reliably returns: a bullet-point summary of the full meeting, a list of action items with an identified owner and deadline for each, the key decisions made during the meeting, and any open questions or follow-up items that remain unresolved.

7.3 RAG Chat Module

To enable conversational interaction with the meeting, the transcript is split into smaller chunks and converted into vector embeddings using HuggingFace's local embedding models. These vectors are stored in ChromaDB. When a user asks a question in the chat interface, the query is embedded and used to retrieve the most semantically relevant transcript chunks from ChromaDB. These chunks, along with the user's question, are passed to Mistral AI, which generates a grounded, context-aware answer — this is the Retrieval-Augmented Generation (RAG) approach.

7.4 User Interface Module

The application's front end is built with Streamlit and enhanced with a custom style.css stylesheet, giving the interface a distinct visual identity beyond Streamlit's default theme. The interface allows users to upload files or paste a YouTube URL, view the generated summary/action items/decisions/open questions in organised sections, chat with the meeting transcript, and download the final report.

7.5 Report Export Module

Once the meeting has been processed, this module compiles the summary, action items, decisions, and open questions into a single structured report, which the user can export as either a PDF file (for formal sharing) or a plain TXT file (for quick reference).

8. Working Methodology

- **Step 1:** The user opens the Streamlit web application and either pastes a YouTube URL or uploads an audio/video file of the meeting.
- **Step 2:** The system extracts the audio and determines whether to use local Whisper (English) or Sarvam AI (Hindi/Hinglish) for transcription.
- **Step 3:** The generated transcript is sent through the LangChain LCEL pipeline to the Mistral AI model.
- **Step 4:** Mistral AI returns a structured summary, action items with owners and deadlines, key decisions, and open questions.
- **Step 5:** In parallel, the transcript is chunked, embedded using HuggingFace Embeddings, and stored in ChromaDB for retrieval.
- **Step 6:** The user can chat with the meeting transcript; each query triggers a retrieval from ChromaDB followed by an answer generated by Mistral AI.
- **Step 7:** The user reviews the complete structured report on the styled Streamlit interface and exports it as a PDF or TXT file.

9. Testing

The application was tested across a variety of scenarios to validate transcription accuracy, summarisation quality, and RAG chat reliability.

9.1 Test Cases

- English meeting recordings of varying lengths were transcribed using local Whisper and checked for transcription accuracy.
- Hindi and Hinglish (code-mixed) recordings were transcribed using Sarvam AI and verified for correctness.
- Generated summaries were reviewed for completeness against the original transcript.
- Extracted action items were checked to confirm owners and deadlines were correctly identified when present in the meeting.
- RAG chat responses were tested with both in-scope and out-of-scope questions to verify grounded, relevant answers.

- PDF and TXT export functions were tested to confirm correct formatting and complete content.

9.2 Observations

The system performed reliably for clearly spoken audio in both English and Hindi/Hinglish. Transcription accuracy was slightly lower for recordings with heavy background noise or multiple overlapping speakers, which is a known limitation of most speech-to-text systems and an area identified for future improvement.

10. Advantages and Limitations

10.1 Advantages

- Low operating cost — built primarily using free and locally-hosted components.
- Strong bilingual support — handles both English and Hindi/Hinglish meetings, unlike most existing tools.
- Saves significant time by automatically extracting summaries, decisions, and action items.
- Interactive RAG chat allows users to query specific parts of a meeting without re-reading the full transcript.
- Flexible input — accepts both YouTube URLs and local audio/video files.
- Custom-styled Streamlit interface offers a clean, professional user experience.

10.2 Limitations

- Transcription accuracy can reduce in noisy environments or with overlapping speakers.
- Dependent on third-party free-tier APIs (Sarvam AI, Mistral AI), which may have rate limits.
- Currently supports English and Hindi/Hinglish only; other languages are not yet supported.
- Local Whisper transcription speed depends on the host machine's hardware.

11. Future Scope

- Adding support for additional Indian and international languages.
- Speaker diarization to attribute statements to specific participants automatically.
- Integration with calendar and task-management tools to automatically assign extracted action items.
- Real-time transcription and summarisation for live meetings.
- Deploying the application on the cloud for multi-user, team-wide access.
- Adding analytics dashboards to track recurring topics and decisions across multiple meetings.

12. Conclusion

The AI Video Meeting Assistant successfully demonstrates how modern speech recognition, large language models, and vector-based retrieval can be combined to automate the process of understanding and acting on meeting recordings. By supporting both English and Hindi/Hinglish transcription, generating structured summaries and action items, and offering a conversational RAG-based chat interface, the tool addresses a genuine gap in existing meeting-assistant solutions — particularly for multilingual users. Built using a cost-effective stack of local and free-tier tools (Whisper, Sarvam AI, LangChain LCEL, Mistral AI, ChromaDB, and HuggingFace Embeddings) and presented through a custom-styled Streamlit interface, this project reflects a practical, end-to-end application of Retrieval-Augmented Generation and natural language processing concepts.

13. References

1. Reference Tutorial — AI Video Assistant using RAG: <https://www.youtube.com/watch?v=xIYJhtL0qbQ>
2. OpenAI Whisper — Official Documentation, <https://github.com/openai/whisper>
3. Sarvam AI — Official Documentation, <https://www.sarvam.ai/>
4. LangChain LCEL Documentation, <https://python.langchain.com/>
5. Mistral AI — Official Documentation, <https://docs.mistral.ai/>
6. ChromaDB — Official Documentation, <https://docs.trychroma.com/>
7. HuggingFace Embeddings, <https://huggingface.co/docs>
8. Streamlit — Official Documentation, <https://docs.streamlit.io/>